

Charles Darwin University

HIT137 – Software Now

Group Assignment 3 – Spot the Difference Game

Semester 1, 2026

Member	Contribution
Ayush Bhusal (s400314)	DifferenceRegion class, OOP structure
Rohan Rai (s401527)	ImageProcessor class, OpenCV alterations
Aayush KC (s401347)	Tkinter GUI, timer, layout
Anurag Deep Silwal (s404653)	ScoreTracker class, error handling

Group DAN/EXT16 | Group 31

Contents

1. Project Overview	3
2. How to Run the Application	3
2.1 Requirements	3
2.2 Installation	3
2.3 Running the Game	3
3. How to Play	4
4. Object Oriented Programming Design	4
4.1 Class 1 – DifferenceRegion (My Contribution)	5
4.2 Class 2 – ImageProcessor	6
4.3 Class 3 – ScoreTracker	6
4.4 Class 4 – SpotTheDifferenceApp	6
5. Image Processing Features	7
6. GUI Features	8
7. GitHub Repository	9
8. Testing	9
9. Output	10
Output when all differences are found	10
Output when all three mistakes are made	11

1. Project Overview

For this assignment, our group built a fully functional Spot the Difference game using Python. I was responsible for the DifferenceRegion class and the overall OOP structure that the rest of the application is built on top of.

The game displays two images side by side. The left image is the original and the right image is a modified version containing exactly five hidden differences. These differences are introduced using natural looking shape overlays. The player finds the differences by clicking on the modified image.

I made sure the program followed clean object oriented design from the start, with each class having one clear job. This made it much easier for the group to work on separate parts without breaking each other's code.

2. How to Run the Application

2.1 Requirements

The following Python libraries need to be installed before running the application:

- Python 3.8 or higher
- opencv-python for image loading and processing
- Pillow for converting images to a Tkinter compatible format
- pyttsx3 for text to speech voice announcements

2.2 Installation

Open the terminal or VS Code terminal and run the following command:

```
pip install opencv-python pillow pyttsx3
```

2.3 Running the Game

Once the libraries are installed, run the game using:

```
python spot_the_difference.py
```

3. How to Play

The game is simple to use. Here is how it works step by step:

- Click Load Image and choose any JPG, PNG or BMP photo from your computer.
- Two images will appear side by side. The left is the original and the right is the modified version.
- Click on the right image wherever you think a difference is hidden.
- A red circle appears on both images when you find a correct difference.
- You are allowed a maximum of 3 wrong clicks per image. If you make 3 mistakes, the canvas locks.
- Click Reveal All to show all unfound differences marked in blue.
- When all 5 differences are found, a voice announcement plays and a popup appears on screen.

4. Object Oriented Programming Design

I structured the application using four classes. Each class has a single clear responsibility which follows good OOP design. The table below shows the main OOP concepts and where they appear in the code.

OOP Concept	Where It Is Used
Inheritance	SpotTheDifferenceApp extends tk.Tk so the App class IS the window
Encapsulation	Each class manages its own data through methods only
Polymorphism	All 5 shape methods share the same signature: (x, y, w, h)
Composition	The App class contains ImageProcessor and ScoreTracker objects
Constructor	__init__ method is defined in all four classes
Class Variables	MAX MISTAKES and NUM DIFFERENCES are shared across all instances

4.1 Class 1 – DifferenceRegion

I personally designed and built this class. It stores all the information about one hidden difference on the image. This includes the position, size, alteration type, and whether the player has found or revealed it. By keeping this data inside its own class, the rest of the program can work with differences cleanly without repeating logic.

```
class DifferenceRegion:
    def __init__(self, x, y, width, height, alteration_type):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.alteration_type = alteration_type
        self.found = False
        self.revealed = False

    def get_center(self):
        # calculate center point of the region
        cx = self.x + self.width // 2
        cy = self.y + self.height // 2
        return cx, cy

    def contains_click(self, click_x, click_y, tolerance=35):
        # check if player clicked close enough to this region
        # using pythagorean distance formula
        cx, cy = self.get_center()
        distance = math.sqrt((click_x - cx) ** 2 + (click_y - cy) ** 2)
        return distance <= tolerance

    def get_rect(self):
        return self.x, self.y, self.width, self.height

    def mark_found(self):
        self.found = True

    def mark_revealed(self):
        self.revealed = True

    def is_active(self):
        return not self.found and not self.revealed

    def __str__(self):
        return f"Region at ({self.x},{self.y}) type={self.alteration_type} found={self.found}"
```

4.2 Class 2 – ImageProcessor

Rohan built this class to handle all image operations using OpenCV. It loads the image, makes an exact copy, and places five non overlapping natural shapes on the copy to create the hidden differences.

Method	Purpose
load image (filepath)	Reads the image file using cv2.imread() and stores it
generate differences()	Places 5 random non overlapping shapes on the modified image
overlaps existing (candidate)	Checks if a new region overlaps any existing region
apply leaf (x, y, w, h)	Draws a green leaf shape using ellipse and alpha blending
apply cloud (x, y, w, h)	Draws overlapping white circles to form a cloud shape
apply brush stroke (x, y, w, h)	Draws a curved tapered paint stroke in a vivid colour
apply ripple (x, y, w, h)	Applies sinusoidal pixel displacement to simulate water
apply sparkle (x, y, w, h)	Draws a star polygon with a glowing halo
get display images (max w, max h)	Scales both images to fit the canvas size
draw circle on image (...)	Draws a red or blue circle at the region position

4.3 Class 3 – ScoreTracker

Anurag built this class to manage the game state. It keeps track of how many differences the player has found, how many wrong clicks were made on the current image, and whether the player is locked out.

Method	Purpose
record find()	Adds 1 to total found and current image found count
record mistake()	Adds 1 to mistakes. Locks the player if 3 mistakes are reached
reset for new image()	Saves current image to history then resets per image counters
is locked()	Returns True if the player has reached 3 mistakes
get mistakes()	Returns the current mistake count for display
get total found()	Returns total differences found across all images
get mistakes remaining()	Returns how many more wrong clicks are allowed
str()	Returns a readable state string for debugging

4.4 Class 4 – Spot_The_DifferenceApp

Aayush built the main application class. It inherits from tk.Tk which means it IS the Tkinter window. It creates all the GUI widgets and connects the ImageProcessor and ScoreTracker together.

Method	Purpose
init ()	Creates window, processor, tracker and builds the UI
build ui()	Creates all Tkinter widgets using grid layout
load image()	Opens file dialog, loads image and generates differences
render images()	Draws scaled images onto both canvases
draw on canvas (canvas, img)	Places a PIL image centred on a canvas widget
redraw circles()	Redraws images with red and blue circles on found regions
on canvas click (event)	Handles player click and checks for hit or mistake
check completion()	Checks if all 5 found, stops timer and shows popup
reveal all()	Marks all unfound regions as revealed in blue
update stats()	Refreshes all stats labels on the screen
speak (message)	Plays voice announcement using pyttsx3 in a background thread
start timer()	Starts the live timer from 0 seconds
tick()	Called every second using Tkinter after() to update timer
stop timer()	Stops the timer and cancels the scheduled callback
format time (seconds)	Converts seconds to a readable format like 1m 23s

5. Image Processing Features

All image changes are made using OpenCV (cv2) as required by the assignment. I designed the five alteration types so they would look natural and not too obvious. The table below explains how each one works.

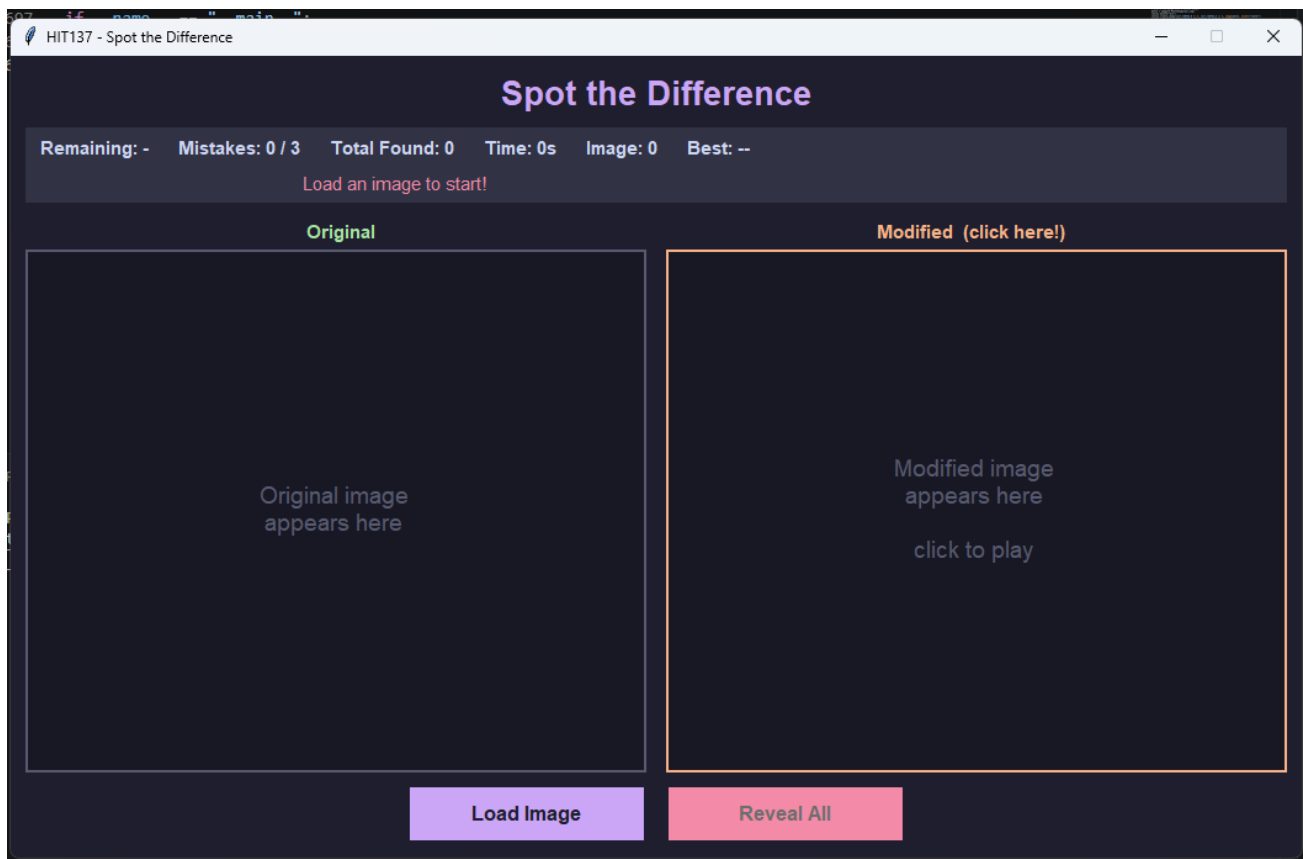
Shape	OpenCV Technique Used
Leaf	cv2.ellipse() to draw a rotated leaf shape on a mask, alpha blending with green colour, cv2.line() for the midrib
Cloud	Multiple cv2.circle() calls on an overlay, cv2.addWeighted() for blending, cv2.GaussianBlur() for soft edges
Brush Stroke	cv2.line() with varying thickness using math.sin(), cv2.addWeighted() for paint blending
Ripple	cv2.remap() with sinusoidal displacement maps to shift pixels, light blue tint added using addWeighted()
Sparkle	cv2.fillPoly() with star polygon coordinates, cv2.circle() for glow halo, cv2.line() for spike rays

Each time an image is loaded, the position and type of each difference is chosen randomly. The placement algorithm makes sure all 5 regions do not overlap each other by using bounding box detection with a 10 pixel margin.

6. GUI Features

Aayush designed the graphical interface using Tkinter. The layout shows both images side by side with a control panel below. Here are the main features of the interface:

- Side by side canvas display for the original and modified images
- Live timer that starts when a new image is loaded and stops when all differences are found
- Stats panel showing mistakes remaining and total differences found
- Load Image button with file dialog supporting JPG, PNG and BMP formats
- Reveal All button to highlight all unfound differences in blue
- Voice announcement played when the game is completed using pyttsx3



7. GitHub Repository

All four group members contributed to the shared repository. I set up the initial project structure and committed the Difference Region class along with the README and GitHub link file. The commit history below shows what each member contributed.

Member	Commit Message	Files Changed
Ayush Bhusal	Initial project setup, Difference Region class, README, GitHub link text file	spot the difference.py, README, github link text file
Rohan Rai	Improve Image Processor shape methods with comments	spot the difference.py
Aayush KC	Add timer and image counter to GUI	spot the difference.py
Anurag Deep Silwal	Add ScoreTracker error handling	spot the difference.py

GitHub Repository URL:

<https://github.com/s400314/HIT137 Assignment3.git>

8. Testing

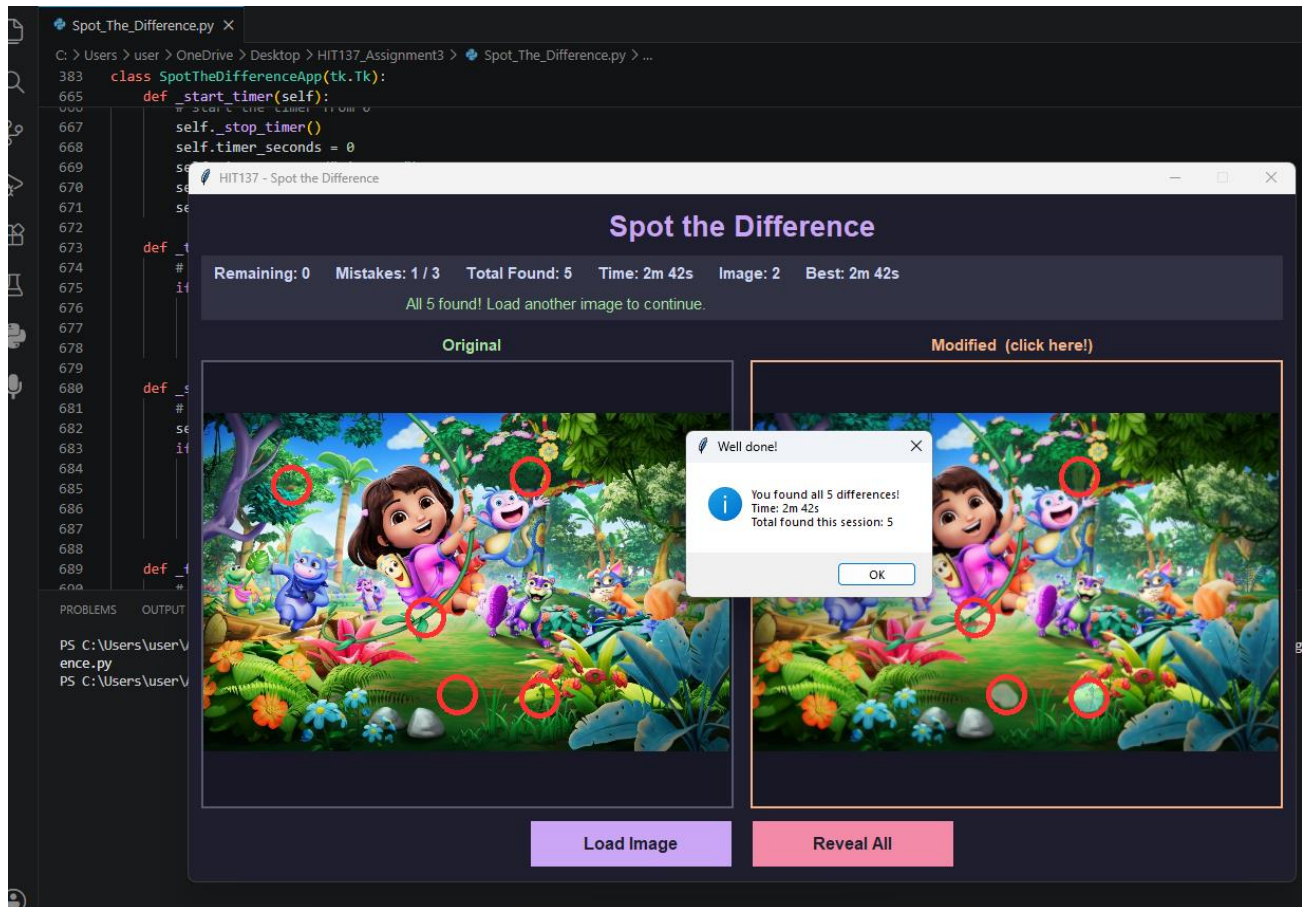
I tested all the main features of the application before submission to make sure everything worked as expected. The table below shows each test case, what I expected to happen, and the result.

Test Case	Expected Result	Status
Load a JPG image	Both images appear side by side	Pass
Load a PNG image	Both images appear side by side	Pass
Load a BMP image	Both images appear side by side	Pass
Click correct difference	Red circle appears on both images, remaining count decreases	Pass
Click wrong area	Mistake counter increases	Pass
Make 3 mistakes	Canvas locks, status message shown	Pass
Find all 5 differences	Voice plays, popup appears, best time recorded	Pass
Click Reveal All	Blue circles on unfound regions, remaining goes to 0	Pass
Load new image after completion	All counters reset, new differences generated	Pass
Timer starts on load	Timer counts up from 0 each new image	Pass

9. Output

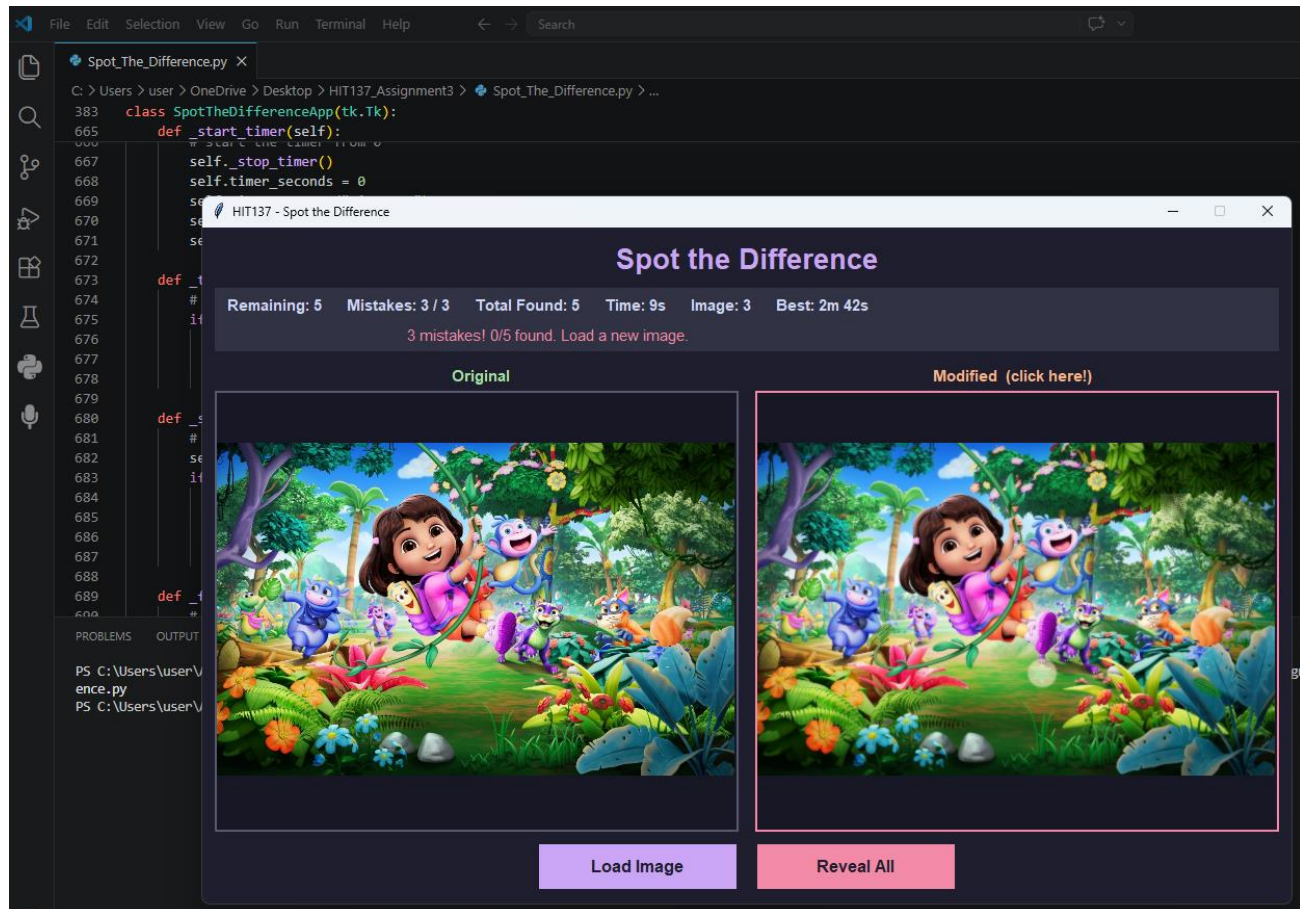
Output when all differences are found

When the player finds all 5 differences, a voice announcement plays and a popup message appears on screen. The timer stops and the best time for that image is saved.



Output when all three mistakes are made

When the player makes 3 incorrect clicks on the same image, the canvas locks and a message is shown on screen. The player needs to load a new image to keep playing.



Thank You!